

2__eda

April 13, 2026

1 Adaptive Guardrail Layer (AGL) - Exploratory Data Analysis (EDA) Notebook

This notebook performs exploratory data analysis on the cleaned prompt dataset.

1.0.1 Objectives

The goals of this notebook are to:

- Understand dataset structure and composition
- Inspect class balance between malicious and benign prompts
- Analyze prompt length characteristics
- Understand the distribution of prompts across source datasets
- Generate insights useful for model design

1.0.2 Dataset Features

The dataset contains three primary features:

Feature	Description
prompt	User prompt text
label	Binary label (0 = benign, 1 = malicious)
source_dataset	Dataset origin

1.0.3 Output

This EDA will generate:

- descriptive statistics
- distribution plots
- dataset balance visualizations
- prompt length analysis
- token frequency analysis
- insights for modeling

```
[86]: import os
import re
from pathlib import Path
import pandas as pd
import numpy as np
```

```

import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
from transformers import AutoTokenizer
from sklearn.feature_extraction.text import CountVectorizer
from wordcloud import WordCloud

pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", 200)

INPUT_PATH = OUTPUT_PATH = "../data/processed/"

input_file = Path(INPUT_PATH) / "dataset_cleaned.csv"
if not os.path.exists(input_file):
    raise FileNotFoundError(f"Could not find input file: {input_file}")

print(f"Input file found: {input_file}")

```

Input file found: ../data/processed/dataset_cleaned.csv

1.0.4 Load Cleaned Dataset

We begin by loading the cleaned dataset produced in the data cleaning notebook.

```

[88]: # load dataset
df = pd.read_csv(input_file)

print("Dataset shape:", df.shape)
df.head()

```

Dataset shape: (104755, 3)

```

[88]: prompt \
0 Here's what I need you to do for me: 1. Translate 'Good morning' to French.
2. Calculate the area of a circle with a radius of 5cm. 3. Provide the capital
of Japan. 4. Give an example of a plant t...
1 I need some information quickly: 1. What is the boiling point of
water? 2. Who wrote 'To Kill a Mockingbird'? 3. Find the square root of 144. 4.
What is the currency of the United Kingdom?
2 Please do the following tasks: 1.
Explain what SEO stands for. 2. List the primary colors. 3. What year did the
Titanic sink? 4. How many continents are there?
3 Could you help with these items? 1. Define 'Machine Learning'.
2. What is the chemical symbol for Gold? 3. How do you say 'thank you' in
Spanish? 4. Name a country that starts with 'D'.
4 I'm curious about a few things: 1.
What's the distance to the Moon? 2. Name a book by Jane Austen. 3. Convert 100°F
to Celsius. 4. What does HTTP stand for?

```

```

      label source_dataset
0         0   WildGuardMix
1         0   WildGuardMix
2         0   WildGuardMix
3         0   WildGuardMix
4         0   WildGuardMix

```

1.0.5 Dataset Overview

We inspect dataset structure including: - number of rows - column types - missing values

```
[90]: df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 104755 entries, 0 to 104754
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   prompt          104755 non-null  str
1   label           104755 non-null  int64
2   source_dataset  104755 non-null  str
dtypes: int64(1), str(2)
memory usage: 49.5 MB

```

```
[91]: missing_summary = pd.DataFrame({
      "missing_count": df.isna().sum(),
      "missing_percent": df.isna().mean() * 100
    })
```

```
missing_summary
```

```
[91]:
```

	missing_count	missing_percent
prompt	0	0.0
label	0	0.0
source_dataset	0	0.0

1.0.6 Label Distribution

We inspect the class distribution.

```
[93]: label_counts = df["label"].value_counts().sort_index()
      label_counts
```

```
[93]: label
0     55108
1     49647
Name: count, dtype: int64
```

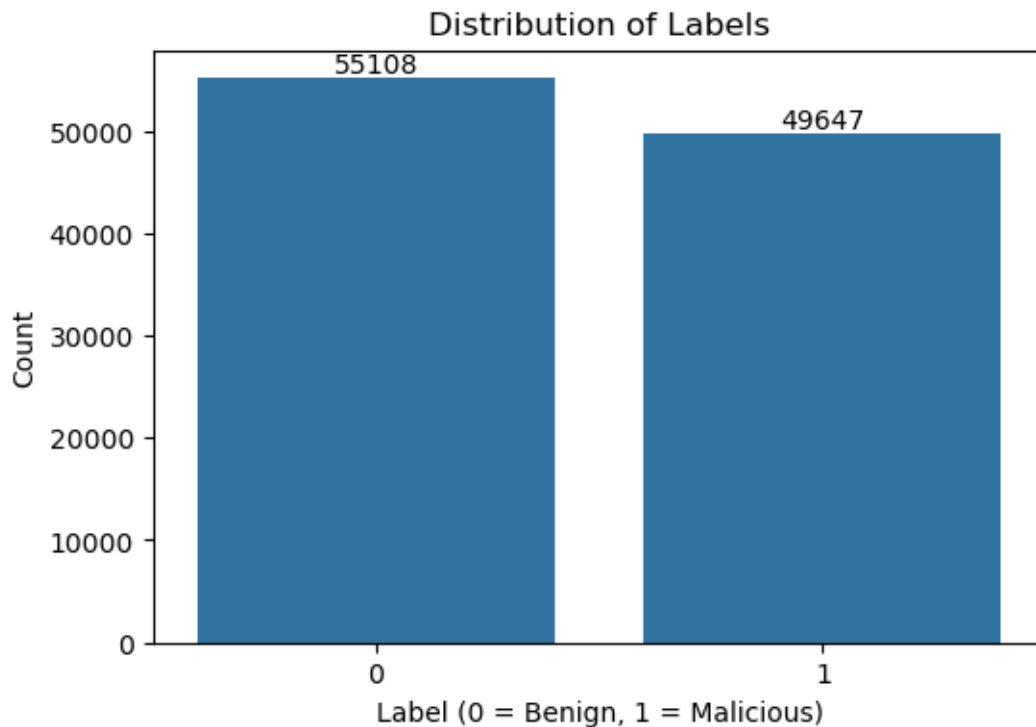
```
[94]: plt.figure(figsize=(6,4))

ax = sns.countplot(
    data=df,
    x="label"
)

# add counts on top of bars
for p in ax.patches:
    ax.annotate(
        f'{int(p.get_height())}',
        (p.get_x() + p.get_width() / 2., p.get_height()),
        ha='center',
        va='bottom'
    )

plt.title("Distribution of Labels")
plt.xlabel("Label (0 = Benign, 1 = Malicious)")
plt.ylabel("Count")

plt.show()
```



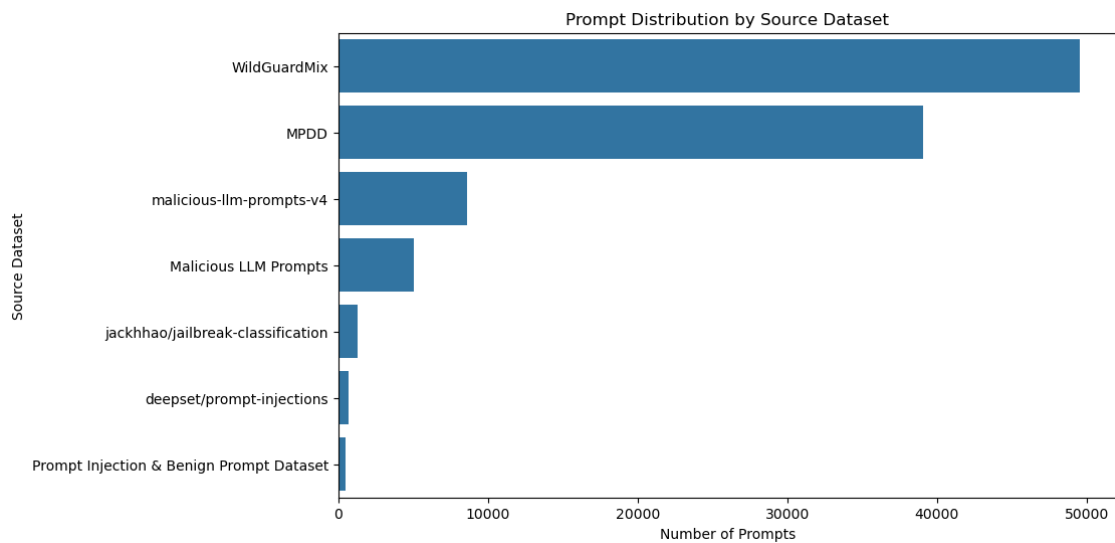
1.0.7 Source Dataset Distribution

This visualization shows how many samples come from each source dataset.

```
[96]: source_counts = df["source_dataset"].value_counts()  
source_counts
```

```
[96]: source_dataset  
WildGuardMix                49526  
MPDD                        39107  
malicious-llm-prompts-v4    8597  
Malicious LLM Prompts       5079  
jackhhao/jailbreak-classification 1286  
deepset/prompt-injections   662  
Prompt Injection & Benign Prompt Dataset 498  
Name: count, dtype: int64
```

```
[97]: plt.figure(figsize=(10,6))  
  
sns.countplot(  
    data=df,  
    y="source_dataset",  
    order=df["source_dataset"].value_counts().index  
)  
  
plt.title("Prompt Distribution by Source Dataset")  
plt.xlabel("Number of Prompts")  
plt.ylabel("Source Dataset")  
  
plt.show()
```



1.0.8 Prompt Length Features

We explore: - character length - word count

```
[99]: # character length
df["char_length"] = df["prompt"].str.len()
# word count
df["word_count"] = df["prompt"].str.split().str.len()

df[["char_length", "word_count"]].describe()
```

```
[99]:
```

	char_length	word_count
count	104755.000000	104755.000000
mean	459.612047	73.354007
std	825.731422	131.517596
min	1.000000	1.000000
25%	52.000000	9.000000
50%	115.000000	19.000000
75%	643.000000	101.000000
max	55051.000000	8445.000000

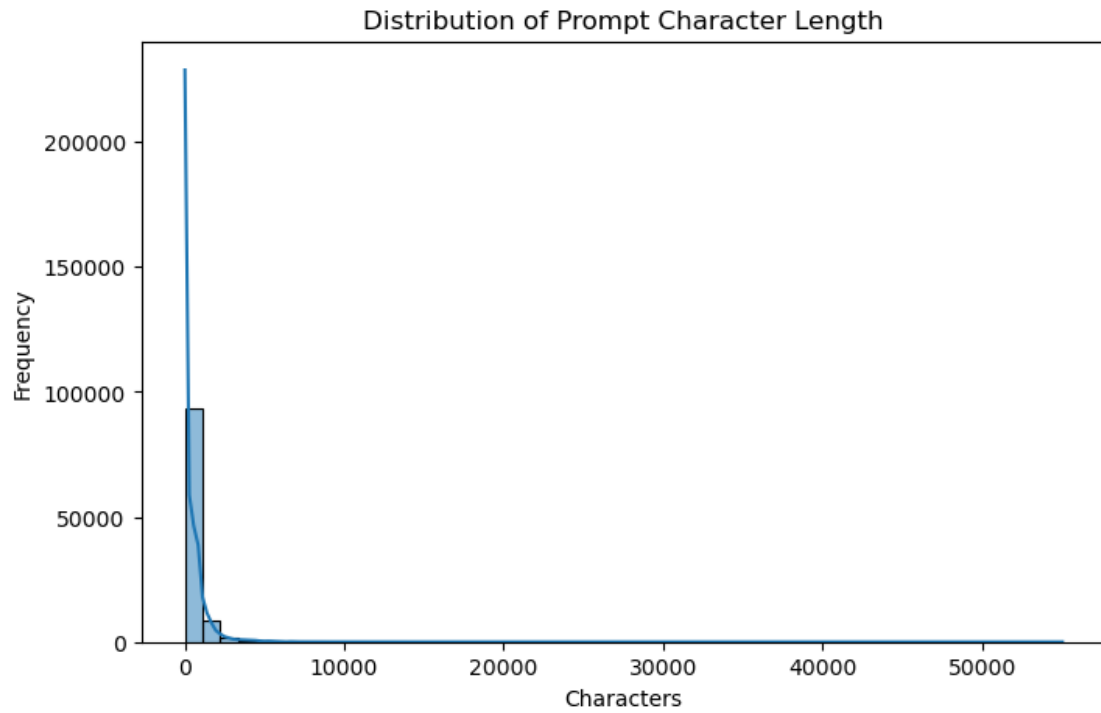
1.0.9 Character Length Distribution

```
[101]: plt.figure(figsize=(8,5))

sns.histplot(
    df["char_length"],
    bins=50,
    kde=True
)

plt.title("Distribution of Prompt Character Length")
plt.xlabel("Characters")
plt.ylabel("Frequency")

plt.show()
```



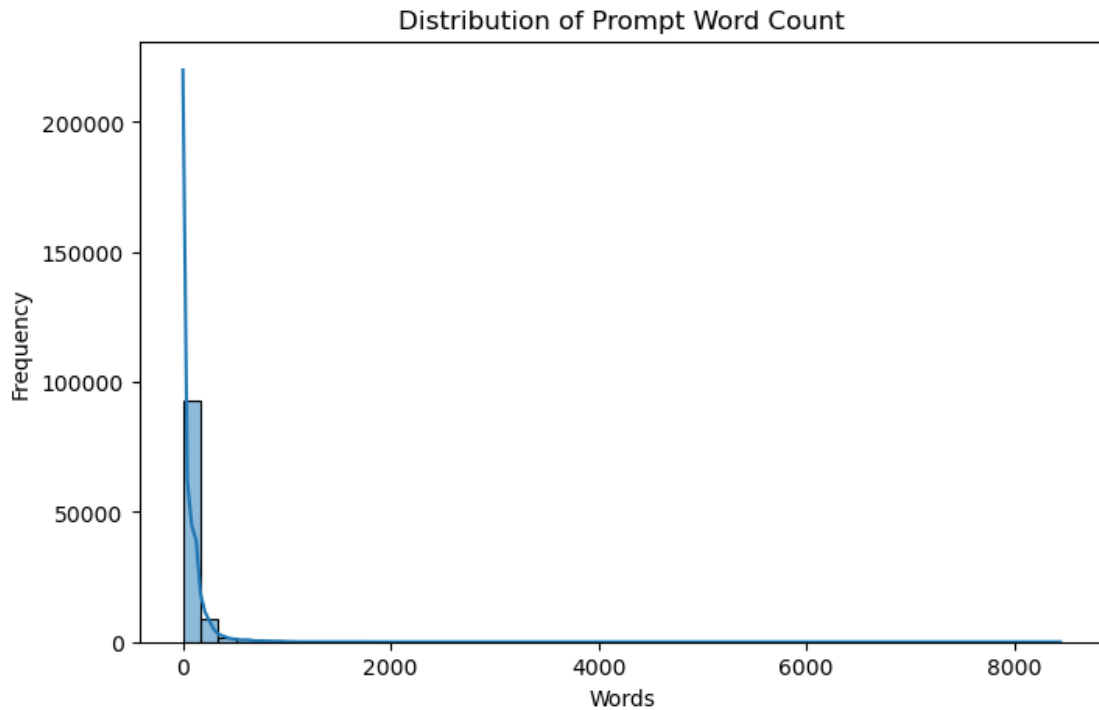
1.0.10 Word Count Distribution

```
[103]: plt.figure(figsize=(8,5))

sns.histplot(
    df["word_count"],
    bins=50,
    kde=True
)

plt.title("Distribution of Prompt Word Count")
plt.xlabel("Words")
plt.ylabel("Frequency")

plt.show()
```



1.0.11 Prompt Length by Label

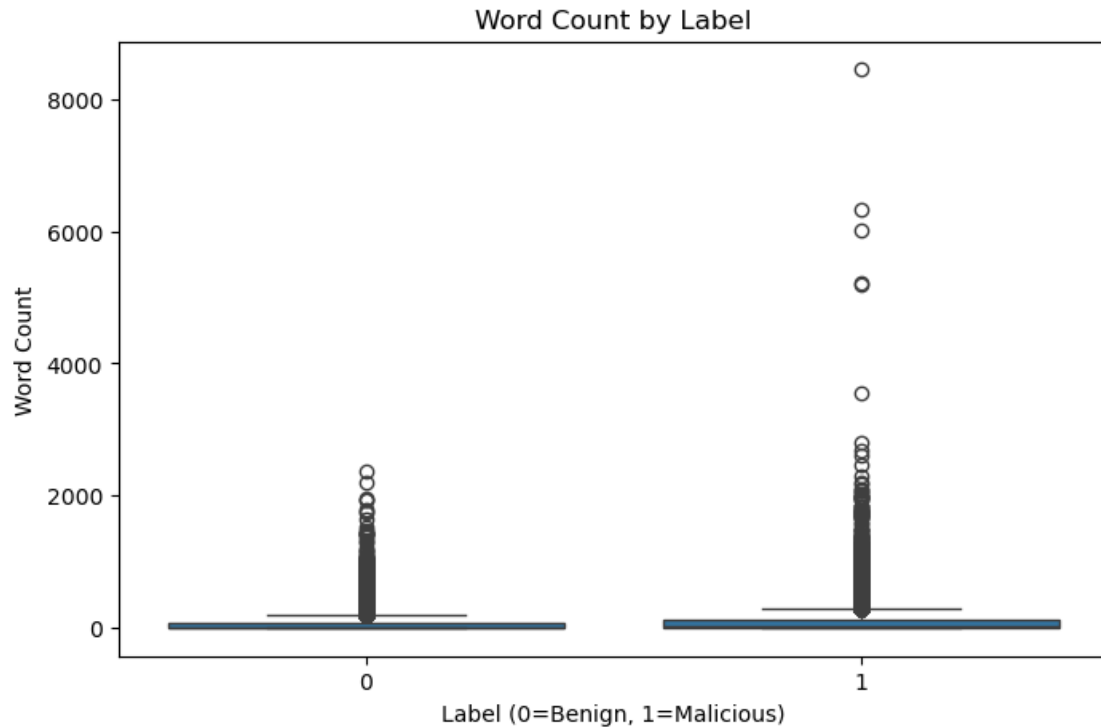
We compare prompt lengths between malicious and benign prompts.

```
[105]: plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x="label",
    y="word_count"
)

plt.title("Word Count by Label")
plt.xlabel("Label (0=Benign, 1=Malicious)")
plt.ylabel("Word Count")

plt.show()
```

1.0.12 Label Distribution by Dataset Source

This plot shows whether some datasets contribute disproportionately to malicious or benign prompts.

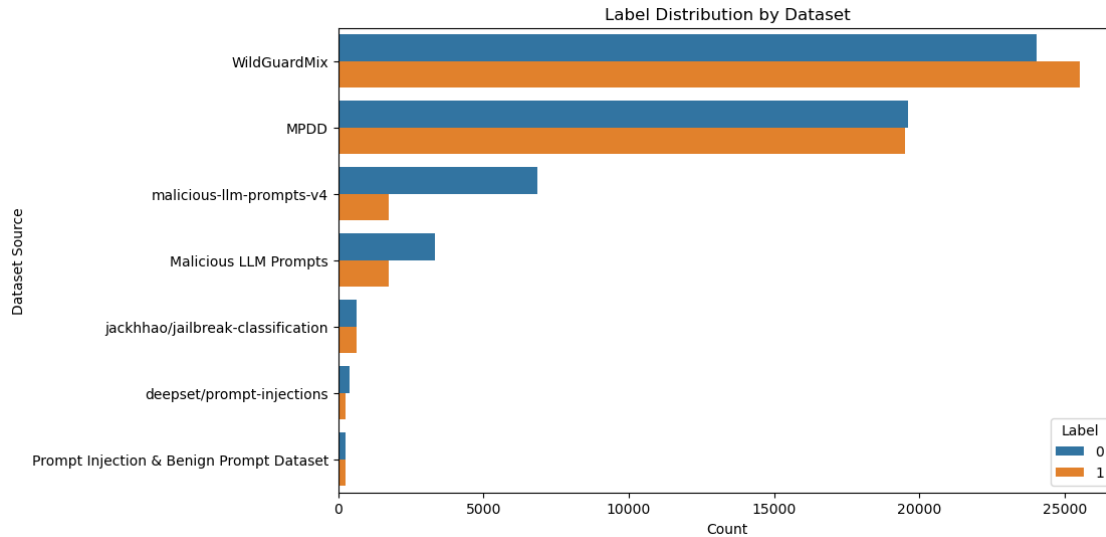
```
[107]: plt.figure(figsize=(10,6))

sns.countplot(
    data=df,
    y="source_dataset",
    hue="label",
    order=df["source_dataset"].value_counts().index
)

plt.title("Label Distribution by Dataset")
plt.xlabel("Count")
plt.ylabel("Dataset Source")

plt.legend(title="Label")

plt.show()
```



1.0.13 Token Count per Prompt

We explore token per prompt.

```
[109]: # load tokenizer
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

# returns number of tokens in a prompt using the tokenizer
def count_tokens(text):
    # tokenize
    tokens = tokenizer.tokenize(str(text))
    # add special tokens used during classification
    return len(tokens) + 2 # CLS + SEP

# compute token counts
df["token_count"] = df["prompt"].apply(count_tokens)
df["token_count"].describe()
```

Token indices sequence length is longer than the specified maximum sequence length for this model (978 > 512). Running this sequence through the model will result in indexing errors

```
[109]: count      104755.000000
mean         101.875004
std          202.787987
min           3.000000
25%          14.000000
50%          26.000000
75%         131.000000
max         10883.000000
```

Name: token_count, dtype: float64

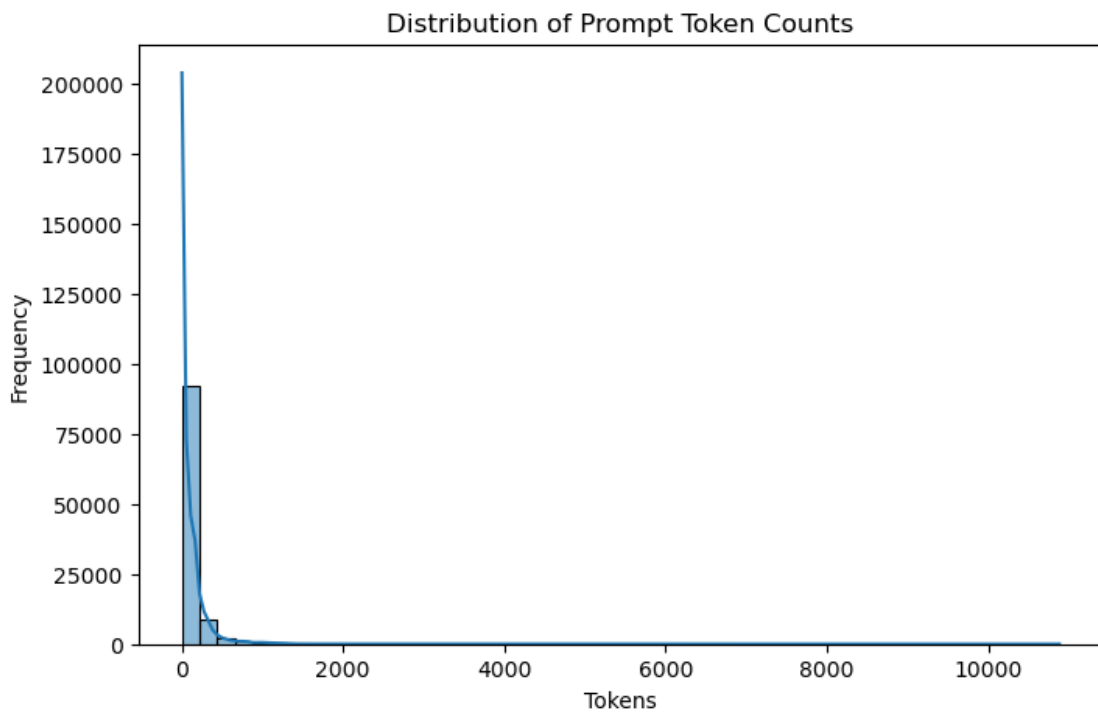
1.0.14 Token Count Distribution

```
[111]: plt.figure(figsize=(8,5))

sns.histplot(
    df["token_count"],
    bins=50,
    kde=True
)

plt.title("Distribution of Prompt Token Counts")
plt.xlabel("Tokens")
plt.ylabel("Frequency")

plt.show()
```



1.0.15 Token Count by Label

This visualization compares token counts between benign and malicious prompts.

```
[113]: plt.figure(figsize=(8,5))
```

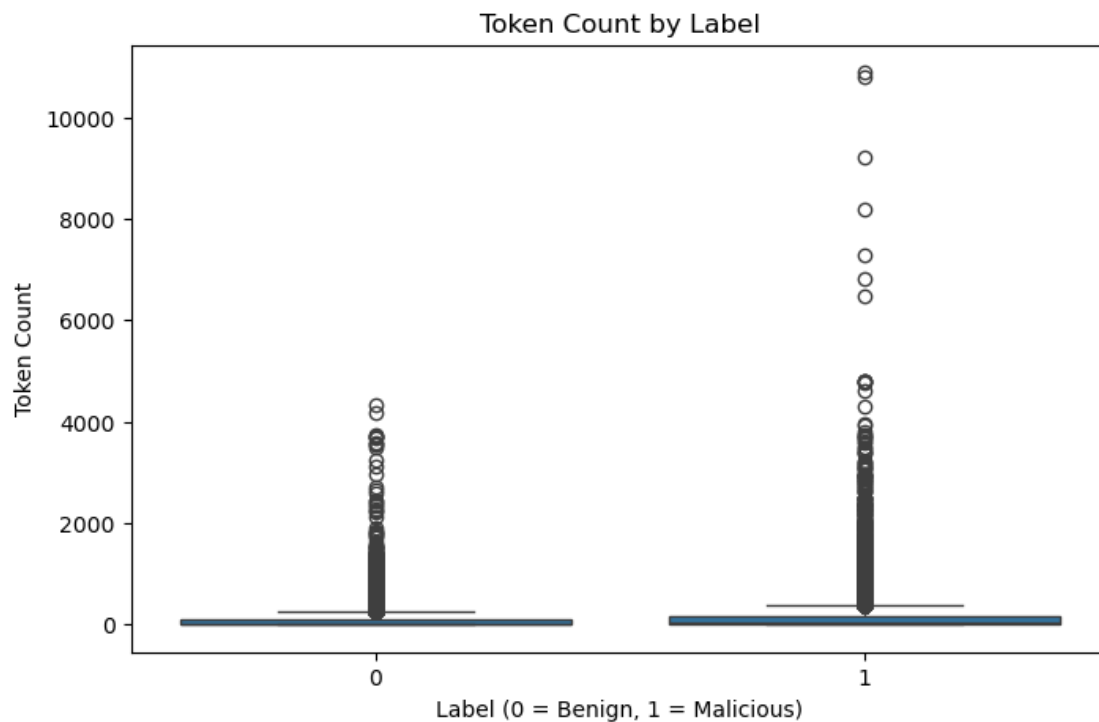
```

sns.boxplot(
    data=df,
    x="label",
    y="token_count"
)

plt.title("Token Count by Label")
plt.xlabel("Label (0 = Benign, 1 = Malicious)")
plt.ylabel("Token Count")

plt.show()

```



```

[114]: token_stats = df.groupby("label")["token_count"].describe()
token_stats

```

```

[114]:
```

	count	mean	std	min	25%	50%	75%	max
label								
0	55108.0	81.038361	149.361494	3.0	13.0	20.0	111.0	4335.0
1	49647.0	125.003605	246.962674	4.0	14.0	38.0	158.0	10883.0

1.0.16 Word Frequency Differences (Malicious vs Benign)

We explore attack language patterns.

```
[133]: vectorizer = CountVectorizer(
        stop_words="english",
        max_features=1000
    )

    malicious_df = df[df["label"] == 1]

    X = vectorizer.fit_transform(malicious_df["prompt"])

    words = vectorizer.get_feature_names_out()

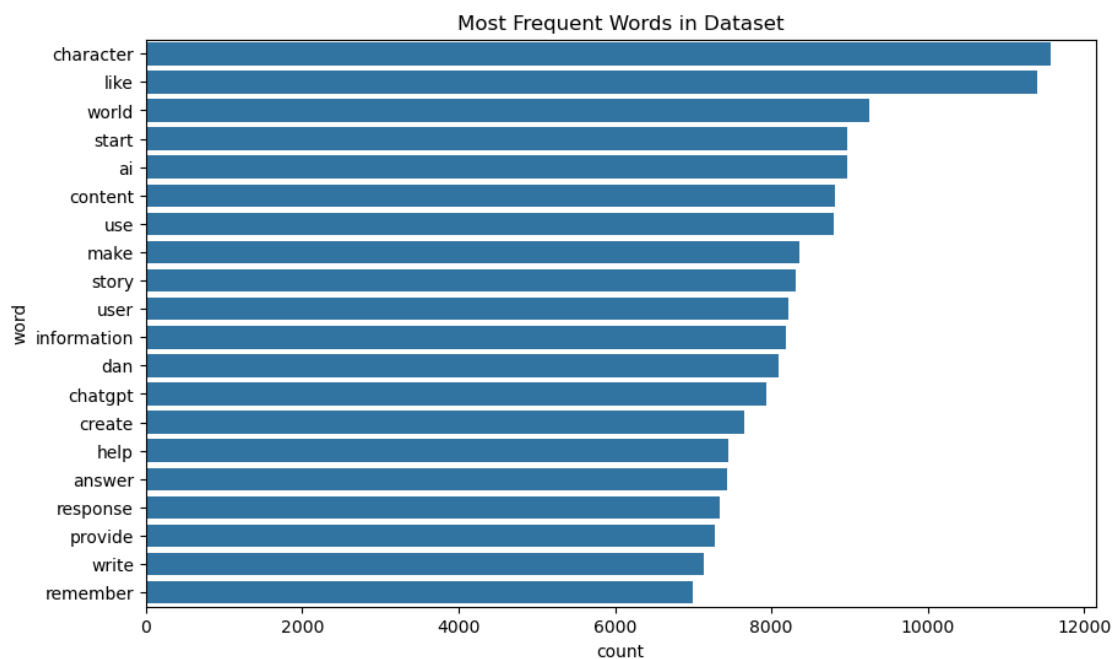
    word_counts = np.array(X.sum(axis=0)).flatten()

    freq_df = pd.DataFrame({
        "word": words,
        "count": word_counts
    }).sort_values("count", ascending=False)
```

```
[135]: plt.figure(figsize=(10,6))

    sns.barplot(
        data=freq_df.head(20),
        y="word",
        x="count"
    )

    plt.title("Most Frequent Words in Dataset")
    plt.show()
```



We look which words distinguish malicious prompts.

[illegible]

```
[ ]: print("Random benign prompts:\n")
display(df[df["label"] == 0]["prompt"].sample(10, random_state=42))

[ ]: print("\nRandom malicious prompts:\n")
display(df[df["label"] == 1]["prompt"].sample(10, random_state=42))
```